UNIVERSITY OF CAMBRIDGE

AI Applied to Finance: Guest Lecture 2

Tutorial

Dr Fabrizio Coiai

Aims of the Tutorial

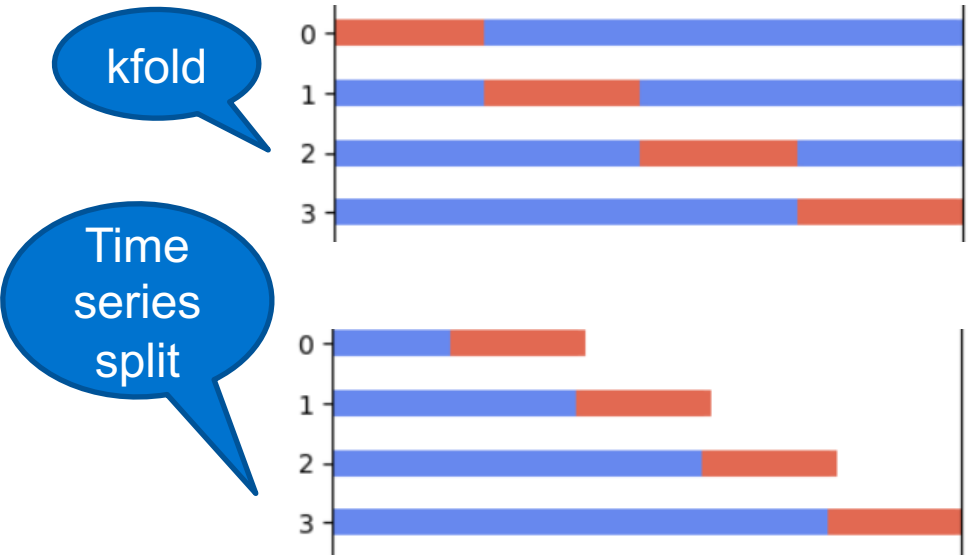
The aims of this tutorial is to familiarize on how to calibrate models using standard package and in particular:

- sklearn
- xgboost

Sklearn packages

- From sklearn we will use:
  - Lasso, Ridge, LinearRegression (sklearn.linear)
  - StandardScaler (sklearn.preprocessing)
  - train_test_split
  - GridSearchCV
  - see TimeSeriesSplit for cross validation with time series data (in the code there is a commented example)

Difference between kfold and TimeSeriesSplit



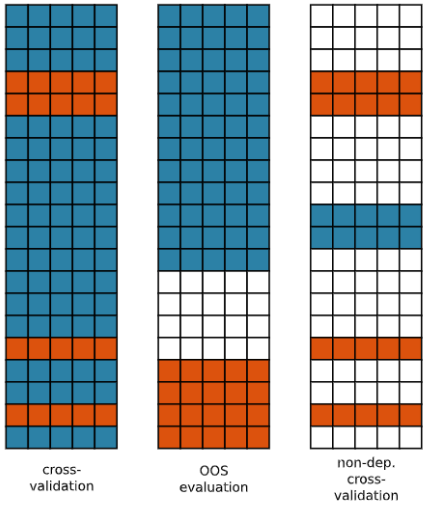


Fig. 1. Training and test sets for different cross-validation procedures for an embedded time series. Rows chosen for training are shown in blue, rows chosen for testing in orange, rows shown in white are omitted due to dependency considerations. The example shows one fold of a 5-fold CV, and an embedding of order 4. So, for the dependency considerations, 4 values before and after a test case cannot be used for training. We see that the non-dependent CV considerably reduces the available training data. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Source: https://scikit-learn.org/stable/auto_examples/model_selection/plot_cv_indices.html#sphx-glr-auto-examples-model-selection-plot-cv-indices-py and Bergmeir et al. (2018)

More on cross validation in time series

Bergmeir, C., Hyndman, R.J. and Koo, B., 2018. *A note on the validity of cross-validation for evaluating autoregressive time series prediction. Computational Statistics & Data Analysis*, *120*, pp.70-83.

- Provide conditions under which leave-one-out and k-fold CV are such that: $\widehat{PE} \xrightarrow{p} PE.$
- From the simulation exercise, it emerges that standard k-fold and leave-one-out are better except when the model is mis-specified
- On an empirical application (yearly sunspot series), it emerges that 5-fold and OOS selection are very close in terms of out of set performance (RMSFE: 2.247 vs 2.281). For 5-fold they select the best model with uncorrelated errors (that pass the Ljung-Box test – max lag = 20 and df=0), for OOS the best model is best overall (also if LB is not applied as first screen)

More on cross validation in time series

Practical Suggestion:

- If you use standard k-fold cross validation, make sure to select the model among the ones with uncorrelated errors as tested with the Ljung-Box methodology (as done in Bergmeir et al. 2018).

- Use time series split and make the split such that you can resemble what you will be doing with the model (e.g.: re-estimation every 4 periods with 4 periods ahead forecasting horizon). If you use data subject to revision or publication lag, make sure you use real time data and take this into account when doing CV.

Housing data - description

Dataset:

- Housing dataset with 1460 observations and 81 columns
- Target variable is *SalePrice*
- Once dropped features with more than 1 missing and deleting the missing row for the remaining feature, we end up with 63 features and 1459 observations
- Dropping also outliers in terms of square feet (top and bottom 1%), we are left with 1427

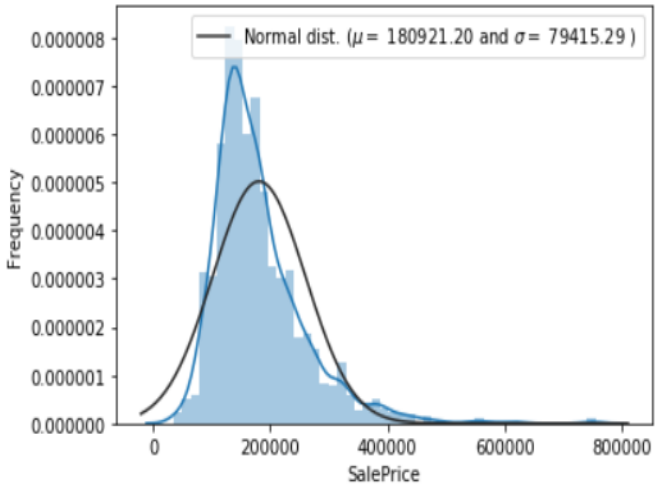
Housing data - transformation

- Converting qualitative to quantitative
  - Some variables are qualitative, e.g.:Street:
    - Type of road access to property
    - Grvl Gravel
    - Pave Paved
  - Need to be mapped to quantitative. How?
  - Dummy var: pd.get_dummies(df)
  - This way we end up with 221 features

Housing data - transformation

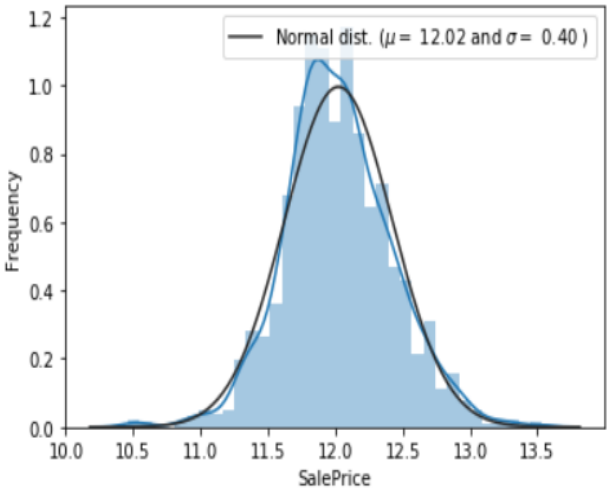
- Should we take the log?

```
mu = 180921.20 and sigma = 79415.29
```



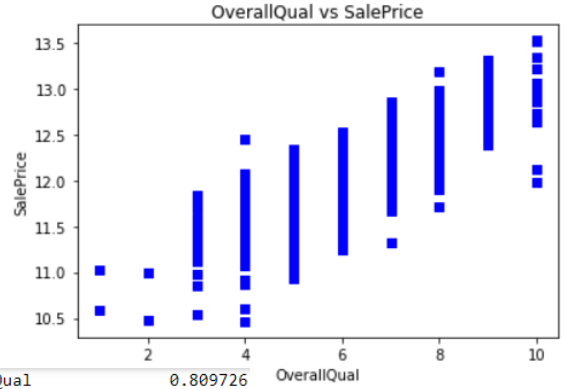

```
1.880940746034036
```

```
mu = 12.02 and sigma = 0.40
```

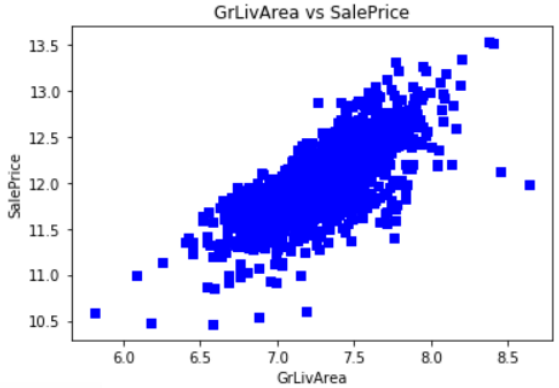


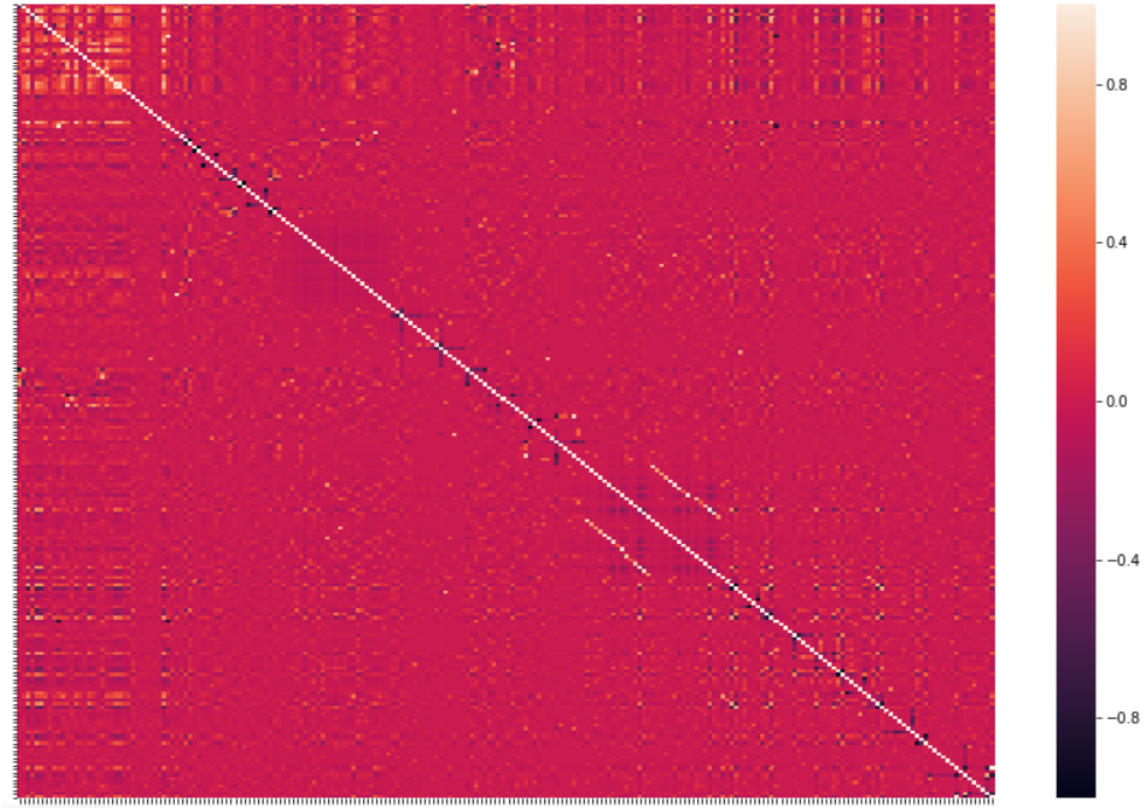

```
0.12096110867730418
```

Housing data - correlation

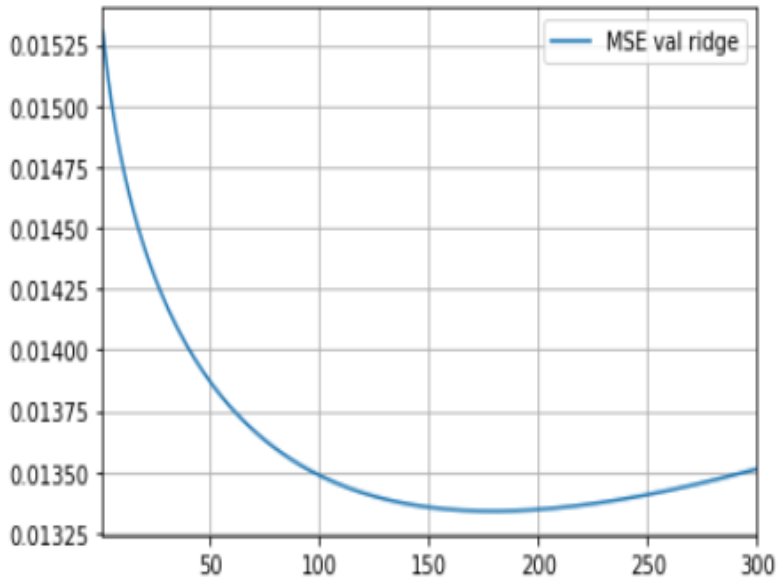


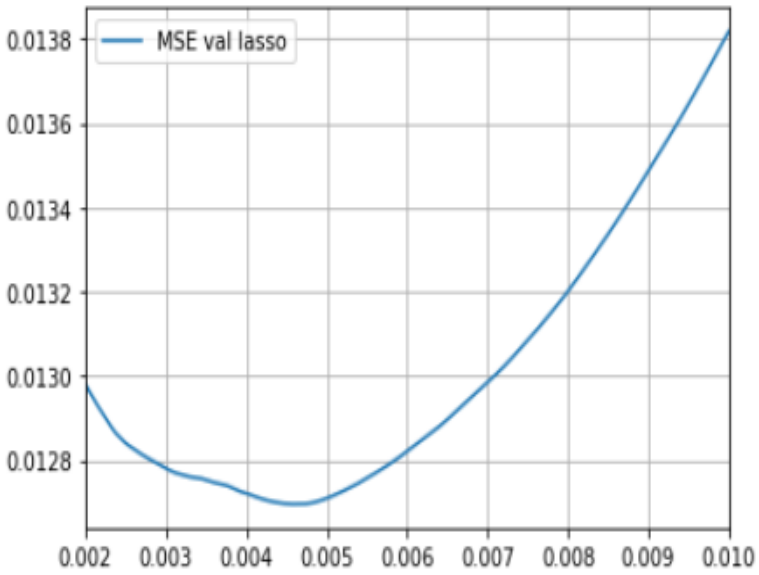

```
OverallQual          0.809726
GrLivArea            0.707538
```





Housing data – hyperparameters lasso and Ridge





xgboost

- link: https://github.com/dmlc/xgboost
- some commonly used hyperparameters:

`eta` [default=0.3, alias: `learning_rate`]
- Step size shrinkage used in update to prevents overfitting. After each boosting step, we can directly get the weights of new features, and `eta` shrinks the feature weights to make the boosting process more conservative.
- range: [0,1]

`gamma` [default=0, alias: `min_split_loss`]
- Minimum loss reduction required to make a further partition on a leaf node of the tree. The larger `gamma` is, the more conservative the algorithm will be.
- range: [0,∞]

`max_depth` [default=6]
- Maximum depth of a tree. Increasing this value will make the model more complex and more likely to overfit. 0 is only accepted in `lossguided` growing policy when tree_method is set as `hist` and it indicates no limit on depth. Beware that XGBoost aggressively consumes memory when training a deep tree.
- range: [0,∞] (0 is only accepted in `lossguided` growing policy when tree_method is set as `hist`)

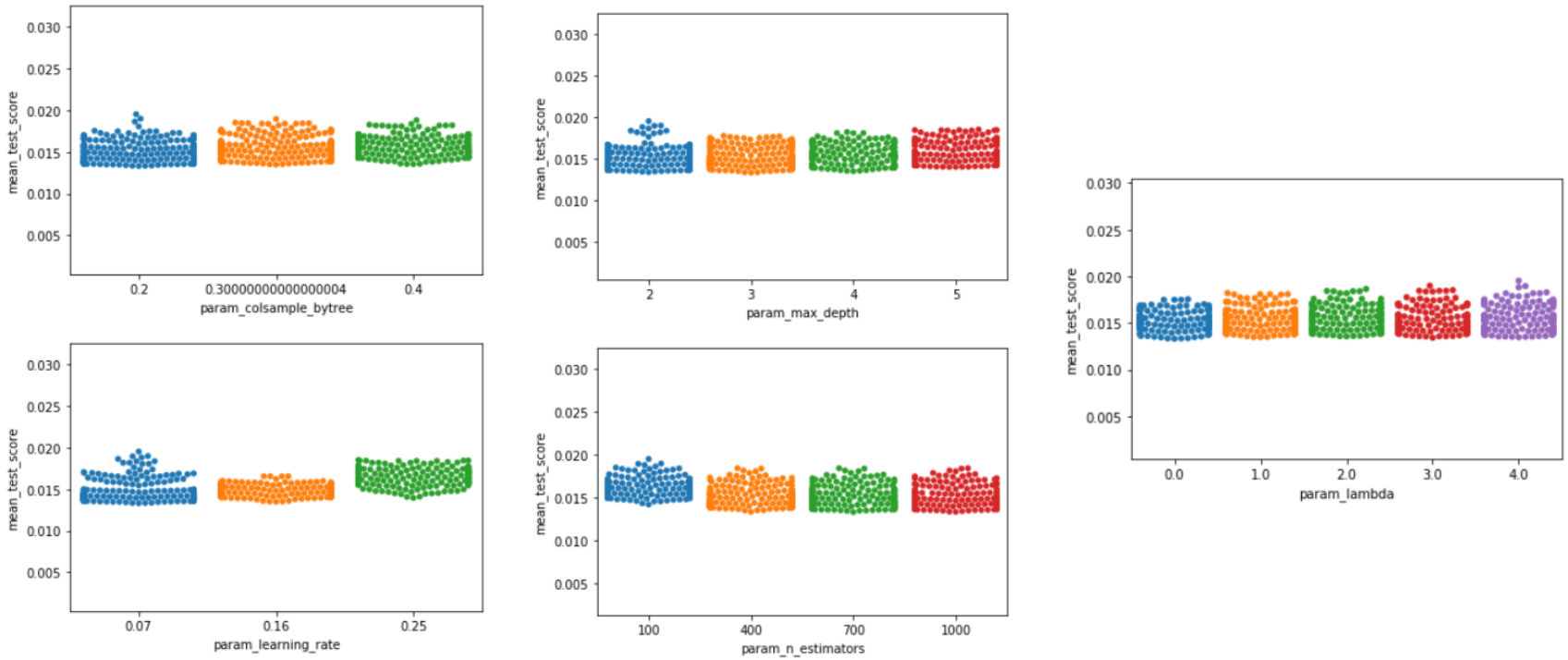
`subsample` [default=1]
- Subsample ratio of the training instances. Setting it to 0.5 means that XGBoost would randomly sample half of the training data prior to growing trees. and this will prevent overfitting. Subsampling will occur once in every boosting iteration.
- range: (0,1]

`colsample_bytree`
- is the subsample ratio of columns when constructing each tree. Subsampling occurs once for every tree constructed.

`lambda` [default=1, alias: `reg_lambda`]
- L2 regularization term on weights. Increasing this value will make model more conservative.

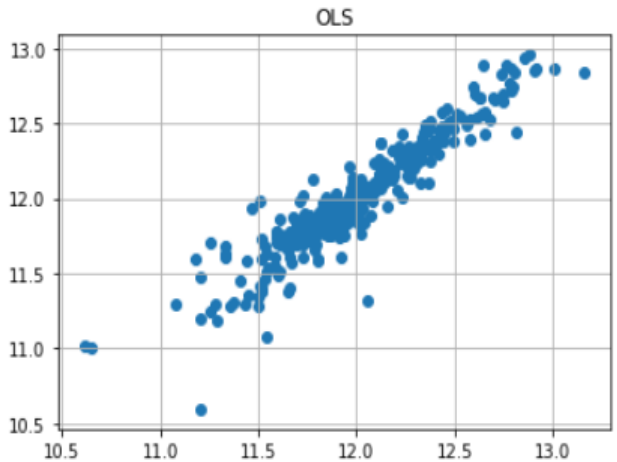
`alpha` [default=0, alias: `reg_alpha`]
- L1 regularization term on weights. Increasing this value will make model more conservative.

Housing data – hyperparameters xgboost

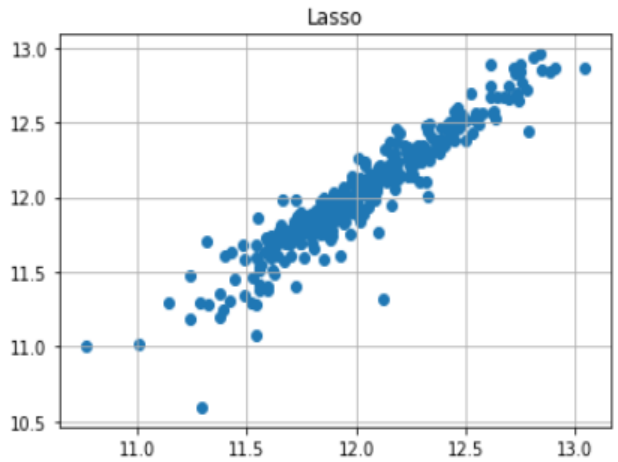


Housing data - results

```
OLS
test
RMSE
0.13188330607848522
train
RMSE
0.08656323960377485
```

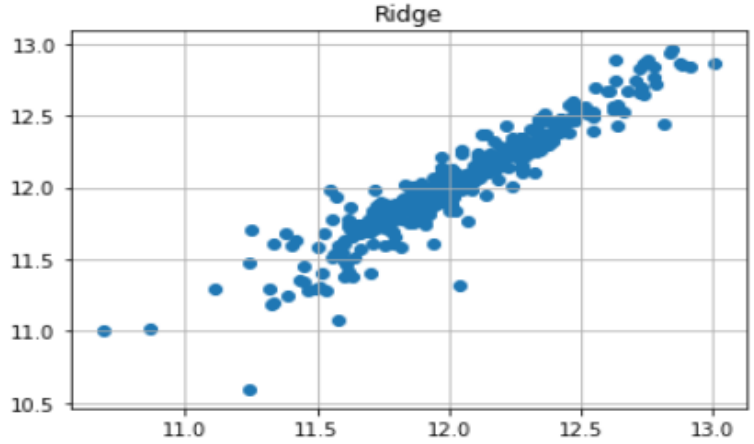



```
Lasso
test
RMSE
0.12173186193361023
train and val
RMSE
0.0985330128085622 0.11267669858861117
```

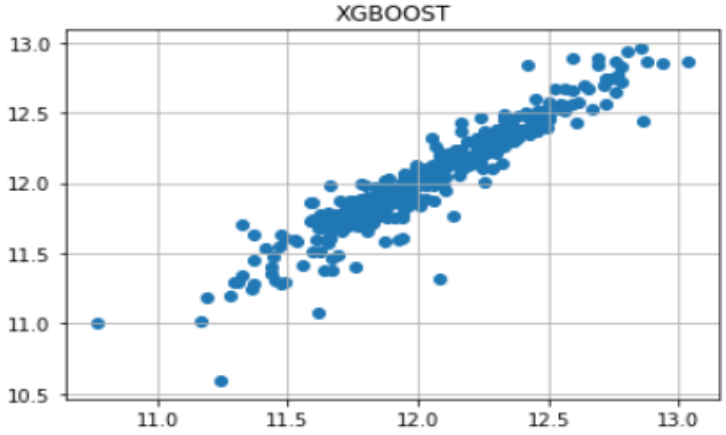


Housing data - results

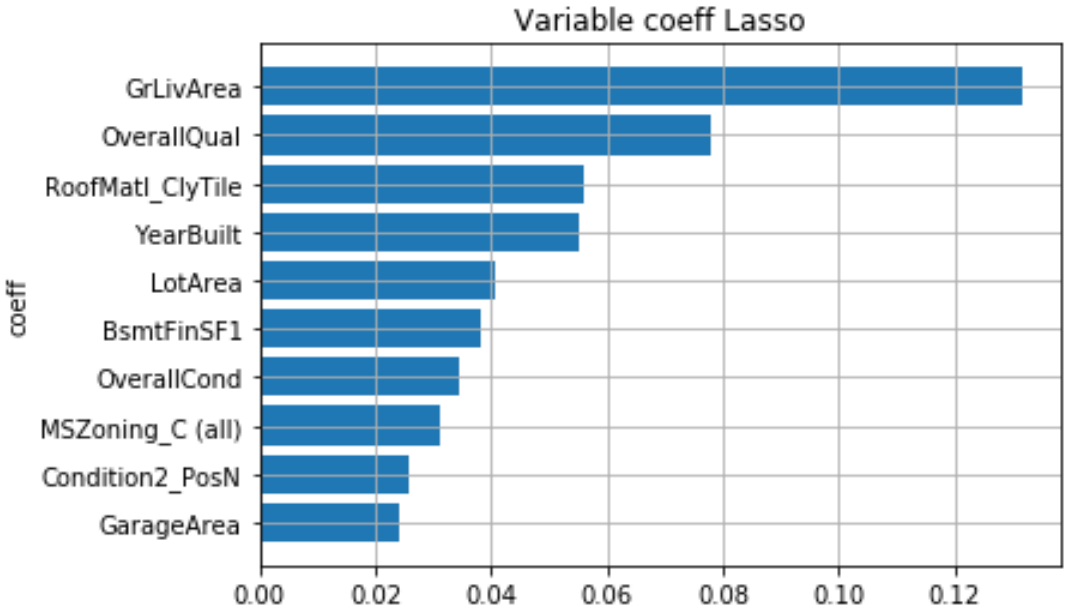
```
Ridge
test
RMSE
0.12519162858077273
train and val
RMSE
0.09195534966520981 0.11549531251507063
```



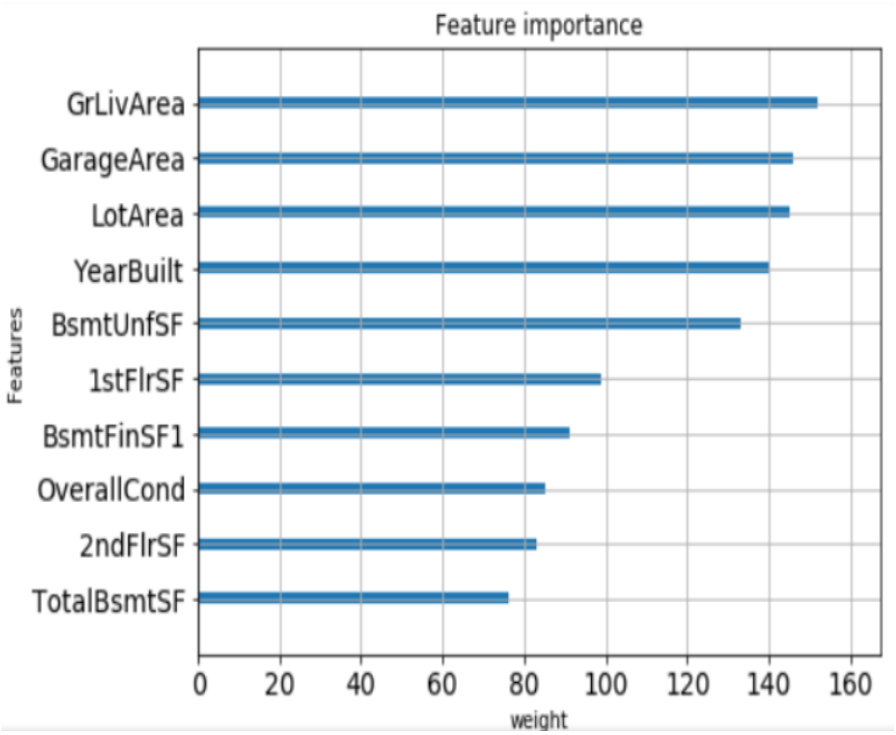

```
XGBOOST
test
RMSE
0.12437189471163096
train and val
RMSE
0.035531033397175456 0.11540657553165969
```

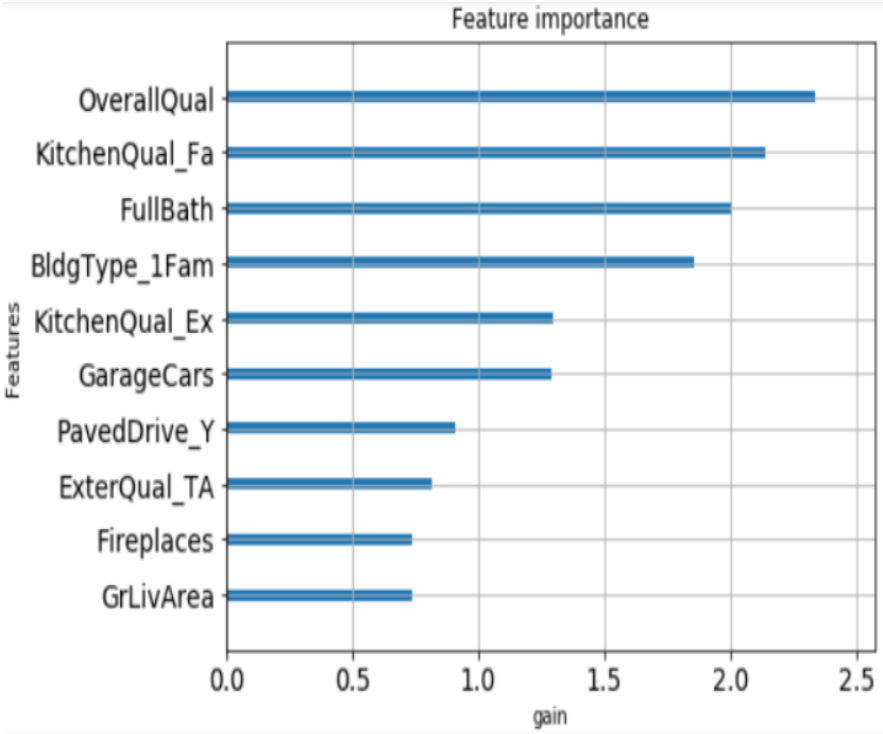


Housing data – feature importance lasso



Housing data – feature importance xgboost





Appendix - feature importance xgboost

- Weight. The number of times a feature is used to split the data across all trees.
- Cover. The number of times a feature is used to split the data across all trees weighted by the number of training data points that go through those splits.
- Gain. The average training loss reduction gained when using a feature for splitting.

Appendix cross validation in time series

Bergmeir, C., Hyndman, R.J. and Koo, B., 2018. *A note on the validity of cross-validation for evaluating autoregressive time series prediction*. *Computational Statistics & Data Analysis*, *120*, pp.70-83.

$$y_t = g(\boldsymbol{x}_t, \boldsymbol{\theta}) + \varepsilon_t, \tag{3}$$

Assumptions

A1 (Stationary nonlinear AR(p) process) A sequence of $\{y_t\}_{t=1}^n$ is represented by the nonlinear AR(p) process as in (3). The nonlinear AR(p) process is assumed to be stationary and ergodic.

A2 (Consistency of the Nonlinear Least Squares) $\hat{\boldsymbol{\theta}}_{-t}$ is a consistent estimator of $\boldsymbol{\theta}$.

A3 (Errors are MDS) The disturbances $\{\varepsilon_t\}$ satisfy the following.

(i) $\{\varepsilon_t, \mathscr{F}_t\}$ form a sequence of martingale differences (MDS) where $\mathscr{F}_t$ is the sigma field generated by $\{\varepsilon_s, y_s; s \leq t\}$. Note that i.i.d errors are MDS.

(ii) $\mathrm{var}(\varepsilon_t|\boldsymbol{x}_t) = \sigma^2$ and $\mathrm{E}|\varepsilon_t|^{4+\delta} < K$ for some $K < \infty$ and $\delta > 0$.

(iii) $\{\varepsilon_t\}$ have absolutely continuous distribution with respect to Lebesgue measure. This assumption is satisfied with errors typically used, including normally distributed errors.

Theorem 1. *Suppose that Assumptions **A1–A3** hold. Then,*

$$\widehat{PE} \xrightarrow{p} PE. \tag{4}$$

Proof. See Appendix. ■

Appendix Ljung-Box test

- Python function: statsmodels.stats.diagnostic.acorr_ljungbox(x, lags=None, boxpierce=False, model_df=0, period=None, return_df=None)
- so in the paper their implementation is with lags = 20, model_df = 0

To run the Ljung Box test by hand, you must calculate the statistic Q. For a time series Y of length n:

$$Q(m) = n(n+2)\sum_{j=1}^{m}\frac{r_j^2}{n-j},$$

Where:

- r_j = the accumulated sample autocorrelations,
- m = the time lag.

We reject the null hypothesis and say that the model shows lack of fit if

$$Q > \chi^2_{1-\alpha,h}$$

$\chi^2_{1-\alpha,h}$ = the value found on the chi-square distribution table for significance level α and h degrees of freedom.

Source: https://www.statisticshowto.datasciencecentral.com/ljung-box-test/

Disclosure

All materials, including video lectures, can be used only and exclusively in relation to course Applications of AI to Finance: alternative data, Sentiment. No download, dissemination of reproduction is allowed.